Virtual Machine in C

Mykyta 'Nick' Khomiakov

March 25, 2026

Contents

1 Abstract **2**

2 Project Setup **4**

1 Abstract

This project presents the design and implementation of a stack-based virtual machine (VM) written entirely in C, conforming to the C11 standard. The machine supports a custom instruction set of 25 opcodes spanning five categories: arithmetic, logic and comparison, control flow, stack manipulation, and register operations. Alongside the VM core, the project includes a purpose-built lexer, a single-pass assembler, and an interactive Read-Evaluate-Print Loop (REPL) shell — all implemented from scratch with no external dependencies beyond the C standard library.

Motivation

The primary motivation for this project was a curiosity-driven investigation into how virtual machines, interpreters, and CPUs work at a fundamental level. Rather than treating a programming language runtime as a black box, the goal was to build one from the ground up — making explicit every design decision that is ordinarily hidden: how instructions are encoded, how operands are passed implicitly through a stack, how control flow is redirected, and how a call stack enables function abstraction.

Architecture Overview

The VM follows a classical *fetch–decode–execute* cycle. On each tick, the instruction pointer (`ip`) reads the next integer from a flat bytecode array (the *fetch* phase). The integer is dispatched through a `switch` statement (the *decode* phase), which routes execution to the appropriate handler function (the *execute* phase). The loop continues until a `HALT` instruction sets the running flag to zero.

The complete runtime state is encapsulated in a single `VM` struct:

- **Operand stack** (`int stack[256]`): The primary working memory. All arithmetic, logic, and comparison operations read their inputs from and write their results to this stack. The stack pointer `sp` always points to the next free slot.
- **General-purpose registers** (`int registers[16]`): Sixteen integer registers (R0–R15), accessible via the `LD` and `ST` instructions. Registers provide a simple means of persisting values across stack operations without re-pushing them every time.
- **Call stack** (`int call_stack[128]`): A dedicated return-address stack, separate from the operand stack, supporting up to 128 levels of nested function calls. The `CALL` instruction saves the current `ip` here before jumping; `RET` restores it.
- **Instruction pointer** (`ip`) and **call stack pointer** (`csp`): Integer indices into the bytecode array and call stack, respectively.

No global variables or hidden singletons exist anywhere in the codebase. Every function receives a pointer to the `VM` struct, making the state explicit and the code straightforward to reason about.

Why Stack-Based?

Most modern physical CPUs are register-based: they shuttle values between a fixed set of named registers (`eax`, `rbx`, `r0`, etc.) before operating on them. A stack-based VM instead uses an implicit operand convention — instructions always read from and write to the top of the stack, so no operand addresses need to be encoded in the instruction stream. This choice was made deliberately for the following reasons:

- **Implementation simplicity.** There is no register-allocation problem to solve. Every opcode knows exactly where its inputs are (the top of the stack) and where to leave its result (back on the top). The interpreter loop reduces to a clean `switch` over a flat integer array with no complex decode logic.
- **Compact bytecode.** Because operands are implicit, most instructions occupy a single integer word. Only instructions that require a literal value — `PUSH`, `JMP`, `JZ`, `JNZ`, `CALL`, `LD`, and `ST` — consume an additional word for that operand. This keeps the bytecode representation minimal and the assembler trivial to implement.
- **Portability.** The VM state is composed entirely of plain `int` arrays and integers. There are no platform-specific intrinsics, no inline assembly, and no OS-level system calls beyond `printf`. The same source compiles and behaves identically on Linux, macOS, and Windows via CMake.
- **Pedagogical clarity.** Stack machines have a direct correspondence with how arithmetic expressions are evaluated in postfix (Reverse Polish Notation) form. The execution model is easy to trace step by step, making this architecture a natural fit for studying how high-level expressions are lowered into a sequence of low-level operations — the same transformation performed by production compilers targeting the JVM or the .NET CLR.

The primary trade-off is instruction count: a stack machine often requires additional instructions (`DUP`, `SWAP`) to reorder operands that a register machine could address directly. This mirrors the philosophy of Reduced Instruction Set Computing (RISC), where a smaller, more uniform instruction set simplifies the processor at the cost of requiring more instructions to perform equivalent work. For a teaching and experimentation platform, this cost is entirely acceptable.

Toolchain: Lexer, Assembler, and REPL

Source text typed at the interactive prompt passes through three sequential stages before execution:

1. **Lexer** (`lexer.c`): Receives a raw line of text. Strips inline comments (any text following a `;` character), uppercases the first token to form the mnemonic, and parses up to two integer operands in either decimal or hexadecimal (`0x...`) notation. Produces a single `Token` struct.

2. **Assembler** (`assembler.c`): Looks up the mnemonic in a static table of 25 entries, each recording the instruction's opcode integer and expected operand count. Emits the opcode word followed by any operand words into a flat `Program` buffer (up to 1024 words). Issues a warning if the operand count does not match the table entry.

3. **VM** (`vm.c`): Receives the populated `Program` buffer, resets a fresh `VM` struct, and enters the fetch–decode–execute loop until `HALT` or a runtime error.

The REPL itself provides four shell commands — `RUN`, `LIST`, `RESET`, and `HELP` — that are intercepted before assembly and never enter the bytecode stream.

Error Handling

The VM detects and reports six categories of runtime error, printing a descriptive message and halting cleanly in each case: stack overflow (operand stack exceeds 256 slots), stack underflow (a pop or binary operation finds fewer values than required), division by zero (`DIV` with a zero divisor), invalid register index (`LD` or `ST` with an index outside 0–15), call stack overflow (`CALL` nested beyond 128 frames), and call stack underflow (`RET` with no matching `CALL`). In every case, the VM sets `running = 0` and returns normally — no crashes, no undefined behaviour.

Separation of Concerns

Each functional category of opcode lives in its own translation unit under `src/operations/`: `maths.c`, `logic.c`, `flow.c`, `registers.c`, and `stack.c`. The central dispatcher in `vm.c` contains only `case` labels; it delegates immediately to the relevant handler. Adding a new opcode requires touching at most three files: the `enum` in `vm.h`, the implementation file, and the mnemonic table in `assembler.c`. No other file needs to change. This modularity was a deliberate architectural goal, ensuring the codebase remains comprehensible as the instruction set grows.

2 Project Setup

Prerequisites

The project used CMake 4.1 and C11 standards for CLang in its setup. However, in order to run the project on any computer, it should have installed dependencies

such as: GCC/CLang with C11 support. GNUMake is required for CMake build on Linux, however CMake can be comfortbaly ran from both Mac and Windows. This document recommends using environments such as Visual Studio for Windows, and CLion for Mac and Linux.

Build

In order to build the project from terminal, this command can be pasted into the terminal with the set directory of the project:

```
gcc -std=c11 -Wall -Wextra -Iinclude \
    src/main.c src/vm.c \
    src/io/lexer.c src/io/assembler.c \
    src/operations/maths.c src/operations/stack.c \
    src/operations/flow.c src/operations/logic.c \
    src/operations/registers.c \
    -o vm
```

In order to run the project either the IDE can handle the runtime, or the project can be ran via the following command:

```
./vm
```

File Directory

```
vm/
├── docs/
│   ├── sections/
│   │   ├── abstract.tex
│   │   └── project_setup.tex
│   │
│   ├── main.tex
│   └── main.pdf
│
├── src/
│   ├── main.c                # Interactive REPL entry point
│   ├── vm.c                  # Fetch-decode-execute loop; push/pop primitives
│   ├── io/
│   │   ├── lexer.c           # Tokenises a line of source text
│   │   └── assembler.c       # Translates tokens into bytecode words
│   └── operations/
│       ├── flow.c            # JMP, JZ, JNZ, CALL, RET
│       ├── logic.c           # EQ, LT, GT, AND, OR, NOT
│       ├── maths.c           # ADD, SUB, MULT, DIV, MOD
│       ├── registers.c       # LD, ST
│       └── stack.c           # PUSH, POP, DUP, SWAP, PEEK, PRINT, HALT
│
├── include/
│   ├── vm.h                  # VM struct, opcode enum, push/pop declarations
│   ├── io/
│   │   ├── lexer.h
│   │   └── assembler.h
│   └── operations/
│       ├── flow.h
│       ├── logic.h
│       ├── maths.h
│       ├── registers.h
│       └── stack.h
│
├── README.md
└── .gitignore
```

Figure 1: File Directory of the project

Separation of Concerns

Each operation category lives in its own translation unit. Adding a new opcode means touching only the relevant file plus a single case in vm.c – the rest of the codebase is unaffected. This was achieved via a divide and conquer approach, where modules that don't have to know about each other – don't know about each other. This separation of concerns allows for a simplified scalable design.